

Parcial Simulacro - BD2

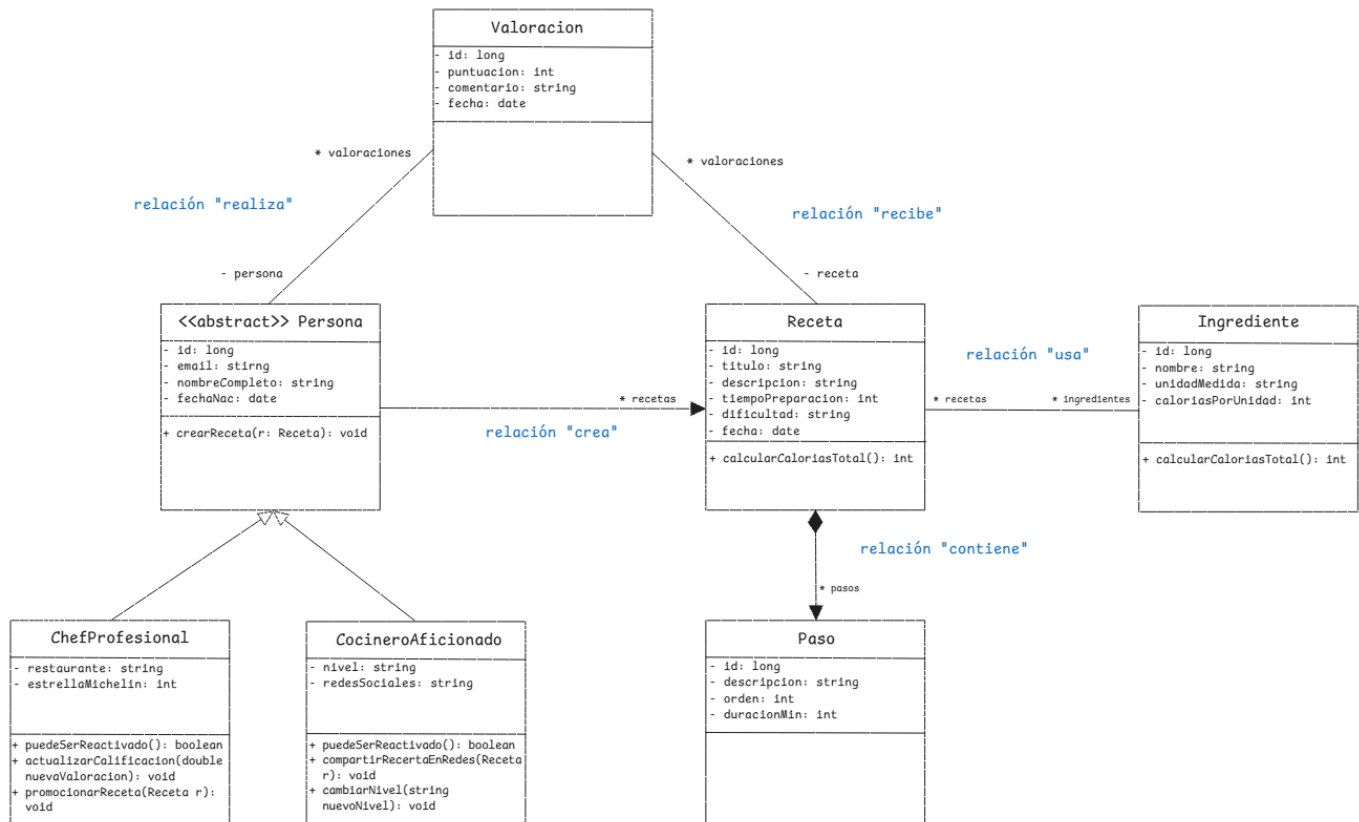
1. Descripción del Modelo

El siguiente diagrama modela una plataforma integral de gestión de recetas de cocina. Esta plataforma permite a distintos perfiles de usuarios interactuar con el sistema culinario.

En el núcleo del modelo, **`Persona`** se define como una clase abstracta de la cual heredan dos perfiles específicos: **`ChefProfesional`** y **`CocineroAficionado`**. Un chef profesional se distingue por estar asociado a un restaurante y poseer una calificación por estrellas, mientras que un aficionado detalla su nivel de experiencia y perfiles sociales.

La interacción principal radica en las recetas. Una **`Persona`** puede crear múltiples **`Recetas`**, pero la autoría de una **`Receta`** pertenece a una sola **`Persona`** (relación "crea", muchos-a-uno). Para estructurar el procedimiento de preparación, cada **`Receta`** se compone estrictamente de uno o más **`Pasos`** (relación "contiene", composición); es decir, los pasos no tienen sentido ni existencia fuera de la receta a la que pertenecen.

El aspecto social de la plataforma permite que una **`Receta`** reciba cero o múltiples **`Valoraciones`**, donde cada **`Valoracion`** es realizada por una **`Persona`**. Finalmente, en la elaboración de los platos, una **`Receta`** requiere el uso de muchos **`Ingredientes`**, y recíprocamente, un **`Ingrediente`** genérico puede aparecer listado en numerosas **`Recetas`** distintas (relación "usa", muchos-a-muchos).



Especificaciones de los Atributos Destacados

- **Persona (abstracta):** *email*: String (no puede repetirse para distintas personas), *nombreCompleto*: String, *fechaNacimiento*: Date.
- **ChefProfesional:** *restaurante*: String, *estrellaMichelin*: int (puede ser 0, 1, 2 o 3).
- **CocineroAficionado:** *nivel*: String ('PRINCIPIANTE', 'INTERMEDIO', 'AVANZADO'), *redesSociales*: String (opcional).
- **Receta:** *titulo*: String (no nulo), *descripcion*: String (opcional), *tiempoPreparacion*: int (minutos), *dificultad*: String, *fecha*: Date.
- **Paso:** *descripcion*: String (no nula), *orden*: int, *duracionMin*: int.
- **Valoracion:** *puntuacion*: int (1-5), *comentario*: String, *fecha*: Date.
- **Ingrediente:** *nombre*: String (único, no nulo), *unidadMedida*: String, *caloriasPorUnidad*: int

Módulo 1: Hibernate / JPA

Pregunta 1 (1 punto)

Al mapear la jerarquía **Persona / ChefProfesional / CocineroAficionado** a tablas relacionales, ¿qué estrategia de herencia de **Hibernate/JPA** utilizaría? Justifique la decisión teniendo en cuenta que las subclases poseen atributos obligatorios propios (por ejemplo, restaurante en **ChefProfesional** y nivel en **CocineroAficionado**) y que el modelo no prevee consultas polimórficas frecuentes sobre todas las personas.

Seleccione la única opción correcta:

- ☐ a. SINGLE_TABLE
- ☐ b. JOINED
- ☐ c. TABLE_PER_CLASS
- ☐ d. No es posible mapear esta jerarquía con JPA porque las subclases tienen relaciones propias.

Pregunta 2 (1 punto)

Defina el mapeo JPA completo de las tres entidades de la jerarquía (**Persona**, **ChefProfesional** y **CocinoAficionado**) según el diagrama. Tenga en cuenta las restricciones de los atributos descritas en la tabla del modelo.

Complete las anotaciones JPA sobre la siguiente estructura de clases, en los renglones indicados con `//`.

```
// @Entity
// @Inheritance(strategy = InheritanceType.JOINED)    @Inheritance(strategy = ...TABLE_PER_CLASS)

public abstract class Persona {

    // @Id; @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // @Column(unique = true)
    private String email;
```

```

        // @Column
        private String nombreCompleto;

        // @Column
        private Date fechaNacimiento;

        // (getters y setters omitidos)
    }

    // @Entity
    public class ChefProfesional extends Persona {
        // @Column
        private String restaurante;

        // @Column
        private int estrellaMichelin;
    }

    // @Entity
    public class CocinoAficionado extends Persona {
        // @Column
        private String nivel;

        // @Column(nullable = true)
        private String redesSociales;
    }

```

Pregunta 3 (1 punto)

Analice la relación "contiene" entre **Receta** y **Paso** y responda las siguientes preguntas:

1. ¿Qué anotaciones se utilizan en cada lado de la relación? ¿Dónde se ubica **mappedBy** y en qué tabla aparece la clave foránea?
2. ¿Qué **FetchType** elegiría en cada extremo de la relación? Justifique.

3. ¿Qué operaciones en **cascada** (y **orphanRemoval**) configuraría desde **Receta** hacia **Paso**? Justifique a partir de la semántica de composición.

```
public class Receta {  
    // ... atributos ya mapeados  
  
    // @OneToMany(mappedBy = "receta", fetch = FetchType.LAZY,  
    // orphanRemoval = true, cascade = CascadeType.PERSIST)  
    //  
    private List pasos = new ArrayList<>();  
}  
  
public class Paso {  
    // ... atributos ya mapeados  
  
    // @ManyToOne(fetch = FetchType.EAGER)  
    //  
    private Receta receta;  
}
```

Módulo 2: Spring Data JPA

Pregunta 4 (1 punto)

Sobre **Spring Data JPA** y los **repositorios**, ¿cuáles de las siguientes afirmaciones son correctas?

Seleccione todas las opciones correctas (puede haber más de una):

- [☒] a. Spring Data JPA es una capa de abstracción sobre JPA/Hibernate que reduce el código repetitivo (boilerplate) de la capa de acceso a datos.
- [☐] b. Al usar Spring Data JPA, Hibernate deja de intervenir: Spring Data genera el SQL directamente sin usar un ORM por debajo.
- [☐] c. Un Query Method derivado por nombre puede resolver cualquier consulta, incluyendo agrupaciones y subconsultas, sin necesidad de @Query.

[V] d. JpaRepository extiende a PagingAndSortingRepository, que a su vez extiende a CrudRepository, agregando operaciones adicionales en cada nivel.

[V] e. @Query acepta tanto JPQL (por defecto) como SQL nativo (con nativeQuery = true).

[V] f. Spring Data JPA genera en tiempo de ejecución las implementaciones concretas de los repositorios definidos como interfaces.

Pregunta 5 (1 punto)

Describa dos situaciones concretas de este modelo donde **Spring Data JPA** simplifica el código en comparación con la implementación manual requerida en **Hibernate puro**. Para cada situación, mencione qué código era necesario escribir antes y qué basta con hacer ahora.

Pregunta 6 (1 punto)

Defina la cabecera (interfaz) del repositorio de una de las entidades mapeadas extendiendo de **CrudRepository**. Indique los parámetros de tipo correctos (entidad e ID).

// Repositorio de la entidad:

```
public interface ChefRepository extends CrudRepository<Chef, Long>
    // Método(s) de consulta (punto siguiente):
    //
    //
}
```

Pregunta 7 (1 punto)

Sobre el repositorio definido en el punto anterior, agregue un método que devuelva todas las recetas que tengan una puntuación promedio de valoraciones mayor o igual a un valor dado. Indique y justifique la estrategia elegida para escribir la consulta (**Query Method derivado por nombre o @Query con JPQL**).

Estrategia elegida y justificación:

Firma del método con la consulta: `@Query("SELECT r FROM Receta r JOIN r.valoraciones v GROUP BY r HAVING AVG(v.puntuacion) >= :valor") List<Receta> findReceta... (@Param("valor") double valor)`

Módulo 3: MongoDB

Pregunta 8 (1 punto)

Suponga que el modelo de la plataforma de recetas se persiste en **MongoDB** con las siguientes dos colecciones:

- **ingredientes:** {_id: ObjectId, nombre: String, unidadMedida: String, caloriasXUnidad: int}
- **recetas:** {_id: ObjectId, titulo: String, tiempoPreparacion: int, dificultad: String, fecha: Date, ingredientIds: [ObjectId, ObjectId, ...]}

Utilizando el **Aggregation Framework de MongoDB**, construya un pipeline que realice las siguientes operaciones en ese orden:

1. Para cada ingrediente, calcular: la cantidad de recetas en las que aparece y el tiempo de preparación promedio de esas recetas.
2. Incluir en el resultado solo los ingredientes que aparecen en más de 3 recetas.
3. Ordenar los resultados por cantidad de recetas de forma descendente.
4. Mostrar únicamente: el nombre del ingrediente, la cantidad de recetas y el tiempo promedio.

Pregunta 9 (1 punto)

Suponiendo que este modelo se va a persistir en MongoDB, seleccione una relación del modelo que convenga modelar como documento embebido y otra que convenga modelar como referencia. Justifique cada una de las elecciones.

Módulo 4: Redis

Pregunta 10 (1 punto)

El equipo de desarrollo de la plataforma de recetas está evaluando usar **Redis** para mejorar la performance de ciertas funcionalidades. Responda las siguientes preguntas:

1. ¿Dónde almacena los datos Redis? ¿Qué implicancias tiene esto en términos de velocidad y de persistencia? ¿Es posible persistir los datos? Explique al menos una de las estrategias disponibles.
2. El equipo quiere implementar un ranking de las 10 recetas más valoradas que se actualice en tiempo real con cada nueva valoración recibida. ¿Qué tipo de dato de Redis usaría para implementarlo? ¿Por qué ese tipo es especialmente adecuado para este caso de uso? Mencione al menos dos comandos relevantes.
3. ¿En qué casos NO conviene usar Redis como único motor de base de datos en una aplicación como esta? Mencione al menos dos limitaciones concretas.